

Software Design

An Introduction

SWE 3633-Software
Design & Architecture

A stylized, dark teal silhouette of a mountain range is positioned in the bottom right corner of the slide, partially overlapping the text area.

Software Design

◆ What?

- What is software design?

◆ Why?

- Why a “good” design matters?

◆ How?

- How can we produce a “good” design?

Software Design - What

◆What?

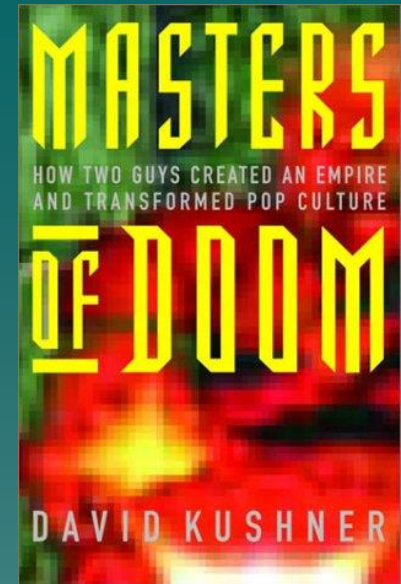
- **Design is the creative process of transforming the problem into a solution.**

Creative?

YES!

ENGINEERING IS CREATIVE

- ◆ **Masters of Doom**
Kushner
- ◆ **Blood Sweat and Pixels**
Schreier
- ◆ **Tommorow, Tomorrow, and Tomorrow**
Zevin



One Definition of Design (from K. Lano)

- ◆ *Design* is the activity of constructing components and organizing their interactions in order to achieve the system that will satisfy the requirements.
 - Components:
 - ◆ Identifying the components
 - ◆ Specifying their functionalities
 - ◆ Specifying any constraints (performance, security, etc.)
 - Interactions:
 - ◆ **Identify and Specify Component relations** (inheritance, aggregation, etc.)
 - ◆ **Identify and Specify Component dependencies** (interfaces, joint responsibilities, sequences of interactions, etc.)

Software Design - Why

◆ Why?

- A good software design
 - ◆ reduces risks in software production,
 - ◆ coordinates development teams to work together orderly,
- 
- A stylized, dark teal silhouette of a mountain range is located in the bottom right corner of the slide, partially overlapping the bottom edge of the text area.

Software Design – Why (cont.)

◆ Why? Cont.

- A good software design
- ◆ makes the system traceable for implementation and testing, and
- ◆ leads to software products that have higher quality attributes.

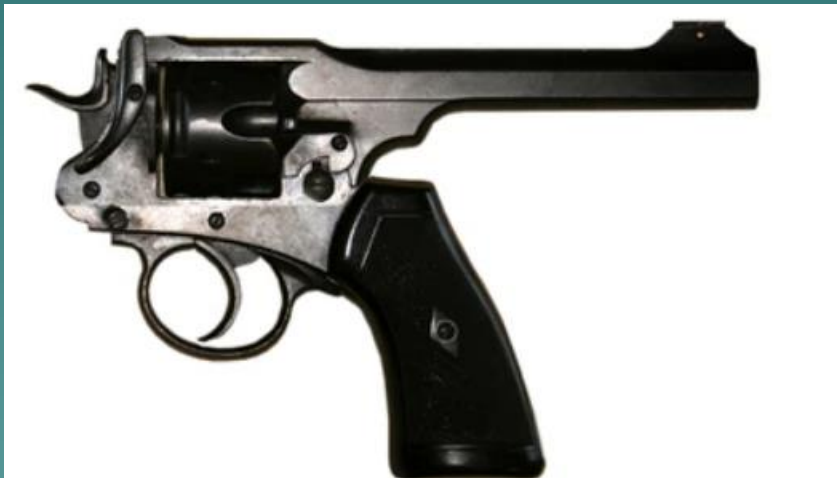
Software Design - How

◆How?

- How can we produce a “good” design?

- ◆Will be discussed in this course

Can you spot "bad" design?



A further website link to bad-design items

From SRS to SDS

- ◆ From requirements specifications to design specifications

SRS → Software Design → SDS

SRS: Software Requirements Specifications

SDS: Software Design Specifications

Software Requirements Specification

- ◆ The SRS is the result of requirement analysis; it records the functional and non-functional requirements that must be met by the software system.

Software Design Specification

- ◆ The SDS describes the software architecture or high-level design and the detailed design of the system.

Software Design Documents

- ◆ **The SDD describes the components of a system, the modules that comprise each component, and the detailed information (such as data, attributes, operations, and algorithms) of each module.**

Software Design Documents (Cont.)

- ◆ From the SDD, the system is then implemented using programming language/s, which is followed by debugging, testing, and maintenance.

Software Design – Two Phases

- ◆ Software **Product Design**

- ◆ Software **Engineering Design**

Software **product design**

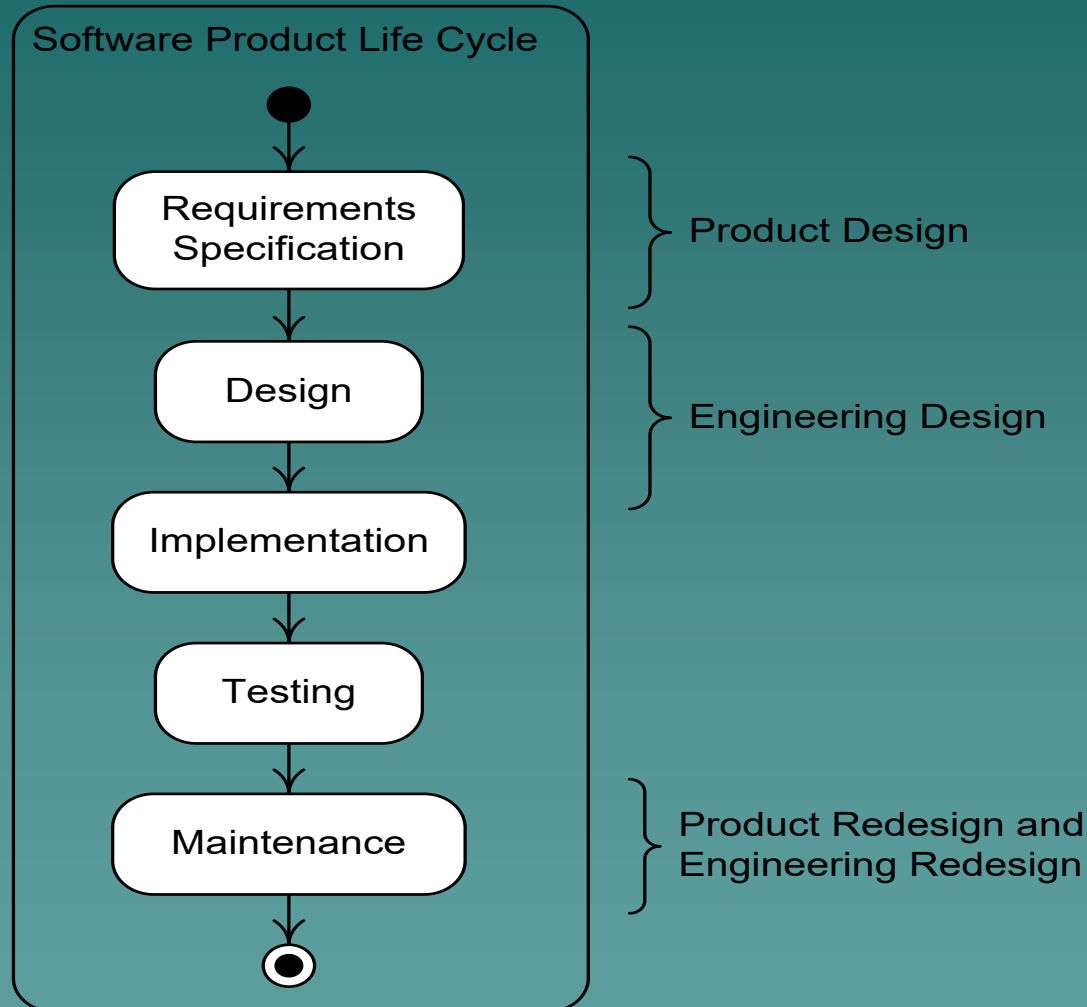
- ◆ is the activity of specifying software **product features, capabilities, and** interfaces to satisfy client needs and desires.

These are not requirements. This comes from requirements.

Software engineering design

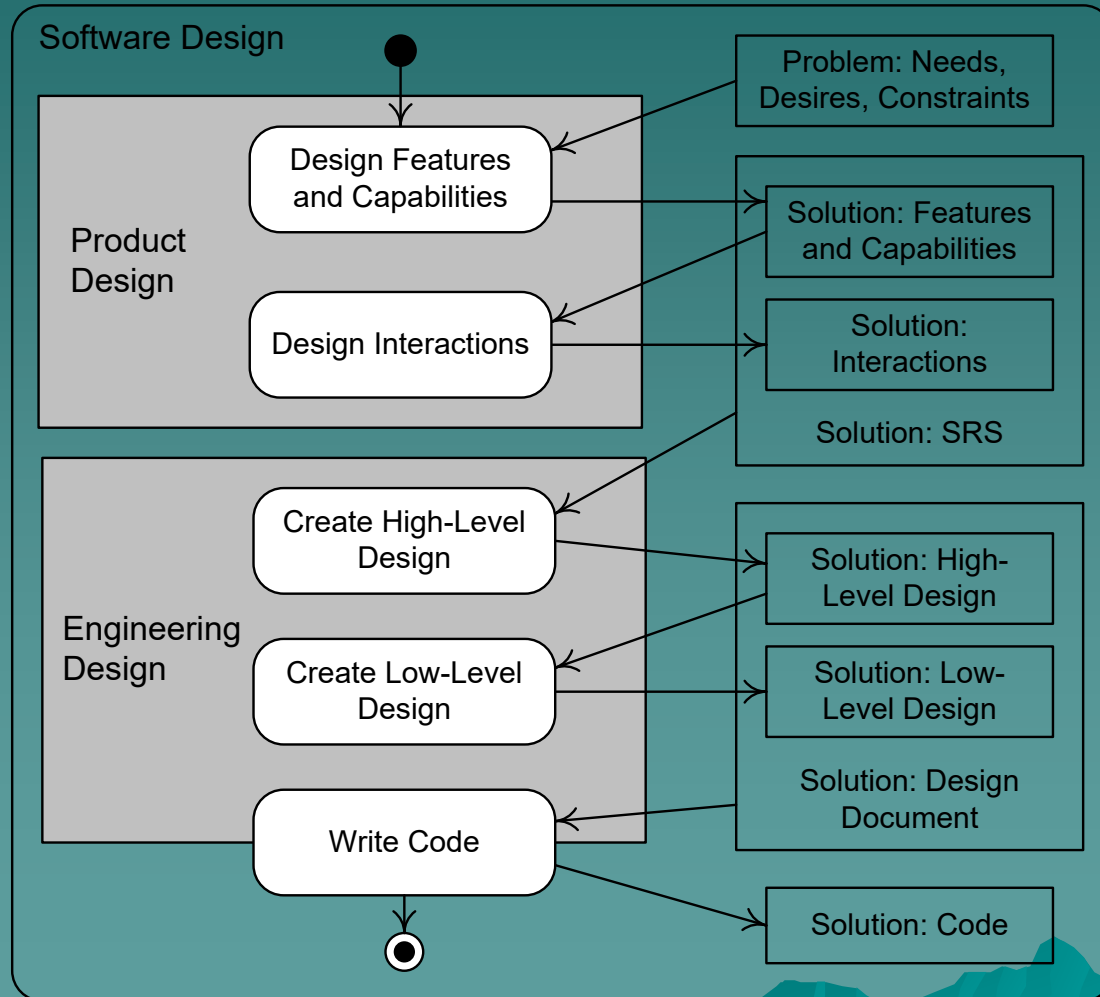
- ◆ is the activity of specifying programs and sub-systems, and their constituent parts and workings, to meet software product specifications.

Software Design Life Cycle



◆ From Page 19 of your textbook

Software Design Problems and Solutions (Page 23)



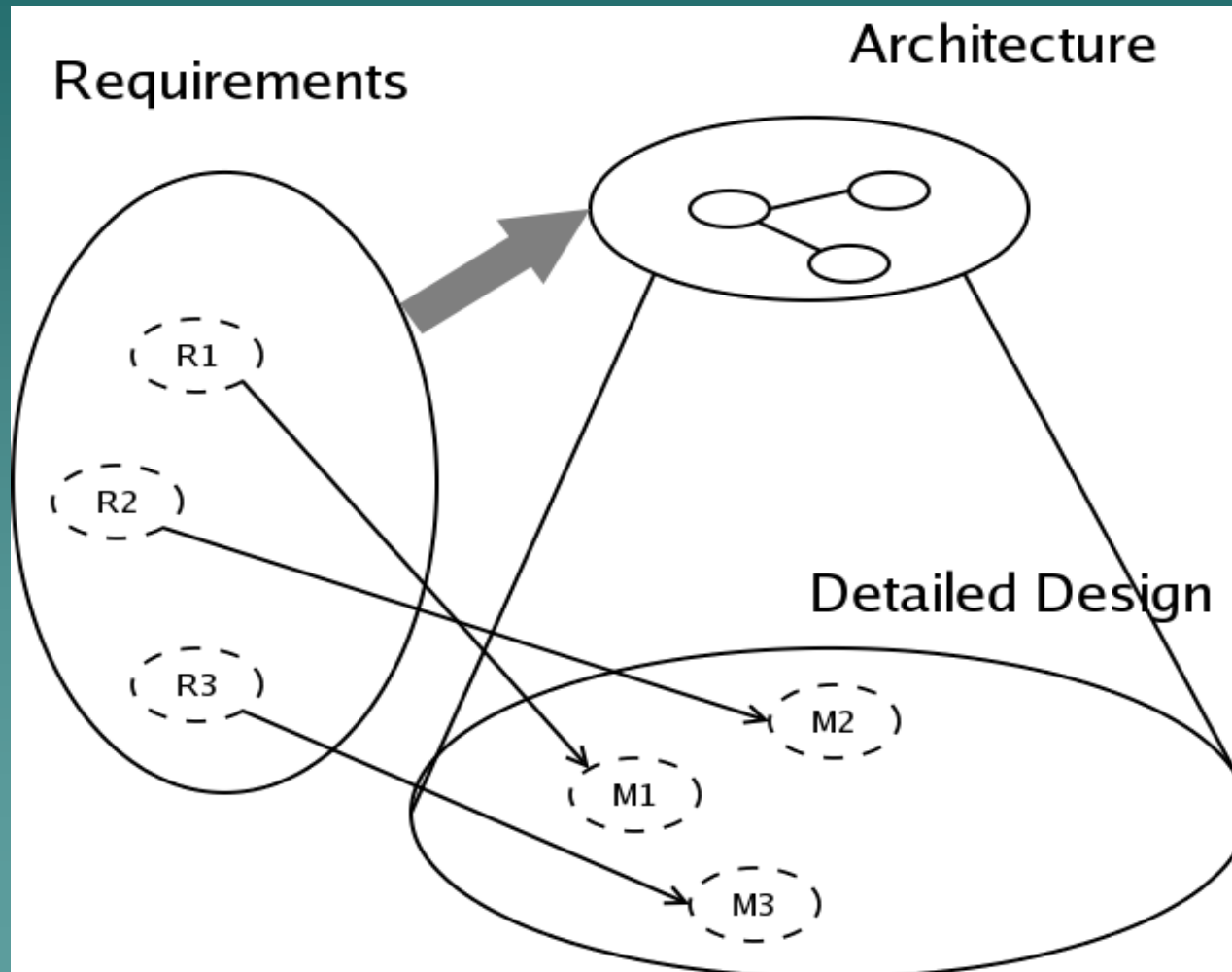
Software Engineering Design

- ◆ **Step-1: Architectural Design**
 - ◆ **Step-2: Detailed Design**
- 
- A stylized, dark teal silhouette of a mountain range is located in the bottom right corner of the slide, partially overlapping the bottom edge of the text area.

Design

- ◆ Start from requirements
- ◆ How is the software going to be structured?
 - What are the components (Modules)
 - How are these components related
- ◆ Two parts
 - Architectural (high-level)
 - Detailed design

Relations of Artifacts



Software Architecture

- ◆ **Structure(s) of system, comprising:**
 - **Software Elements**
 - **Their externally visible properties**
 - **Relationships among elements**
- ◆ **Every system has an architecture**
- ◆ **Multiple structures**
 - **multiple ways of organizing elements**
- ◆ **External properties of modules**

Architectural Tactics

- ◆ **Solve smaller, specific problems**
A wiki page to unix philosophy.
- ◆ **Do not affect overall structure of system**

What is a tactic?

discuss a key tool of Architectural Tactics.

A stylized, dark teal silhouette of a mountain range is positioned in the bottom right corner of the slide, extending from the right edge towards the center.

Detailed Design

- ◆ **Refine Architecture and match with Requirements**
 - ◆ **How detailed ?**
 - ◆ **How formal ?**
 - ◆ **Maybe different level of detail for different views**
- 
- A stylized, dark teal silhouette of a mountain range is positioned in the bottom right corner of the slide, adding a decorative element to the background.